



IAD-014 Redes Neurais

aula 2

Fundamentos de RNAs

Claudio Pinhanez

(a partir de materiais criados por Marlon Sproesser Mathias)



Claudio Pinhanez

40 anos como cientista de IA
professor, inovador, consultor

CIAAM – Centro de IA e Aprendizado de Máquina da USP
Professor Visitante (a partir de 1º de junho, 2026)

2009-2025 IBM Research Brasil
Principal Research Scientist

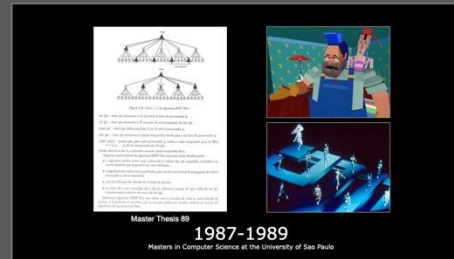
2020-2025 C4AI – USP-IBM-FAPESP
Vice-Diretor

1999-2009 IBM Research T. J. Watson
Research Scientist

1987-1993 University of São Paulo
Professor Assistente

1993-1999 PhD MIT Media Laboratory
1985-1987 Mestrado em Computação USP
1982-1985 Bacharel em Matemática USP





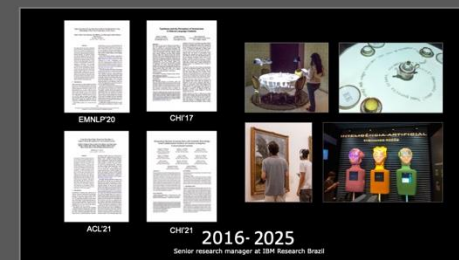
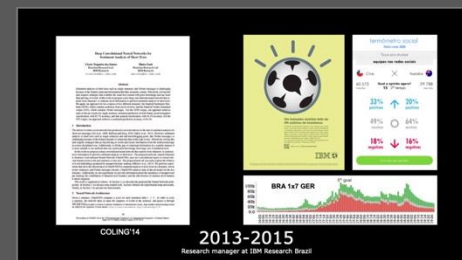
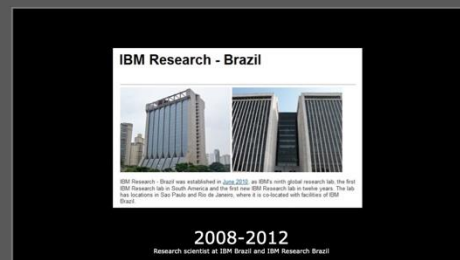
Bacharelado e Mestrado, Universidade de São Paulo

Universidade de Osaka



PhD, MIT Media Laboratory

IBM Research Watson



IBM Research Brasil

Universidade de São Paulo

Calendário e Trabalhos

Julho: ~~30~~

Agosto: 06, 13, 20, 27

Setembro: 03, 10, 17, 24

Outubro: 01

Horário: 19:30 – 22:30 (15min break)

	enunciado	entrega
Trabalho 1:	06 de agosto	20 de agosto
Trabalho 2:	27 de agosto	10 de setembro
Trabalho 3:	17 de setembro	01 de outubro

Média final: $(2 \cdot \text{participação} + 2 \cdot T1 + 3 \cdot T2 + 3 \cdot T3) / 10$

Uso de IA nos trabalhos:

O uso extensivo de IA nos trabalhos não é permitido nem aconselhável.

Erros estúpidos e básicos, típicos de IA, serão altamente penalizados.

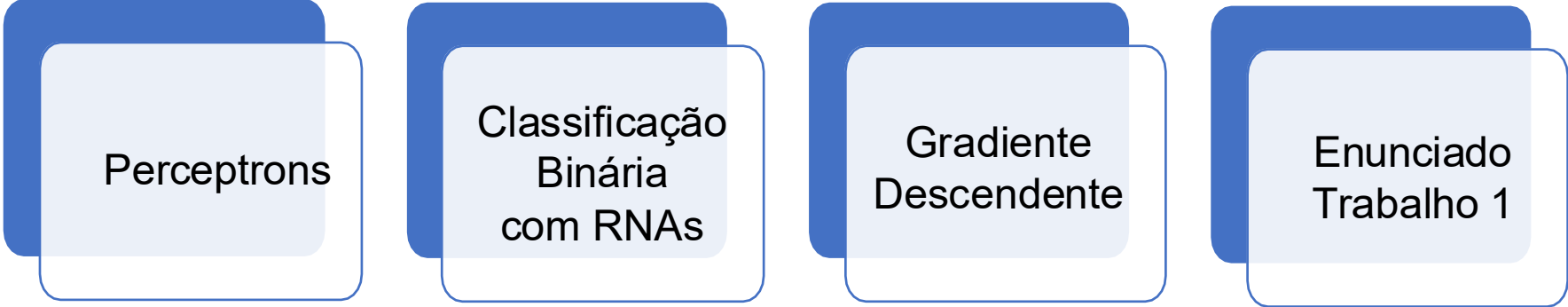
O aluno/a deverá ser capaz de responder integralmente pelo que foi submetido sob pena de anulação.

Bonus: Boom or Doom

O estado da IA em 2026

Boom or Doom?

Aula 2 – Fundamentos de RNAs



Perceptrons

Classificação
Binária
com RNAs

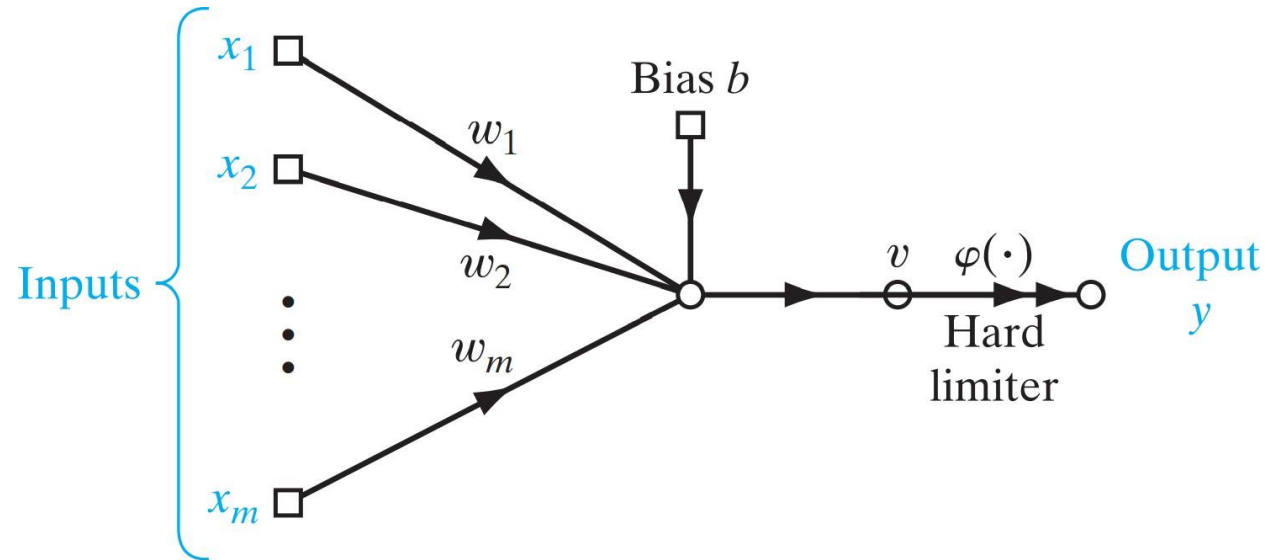
Gradiente
Descendente

Enunciado
Trabalho 1

Perceptrons

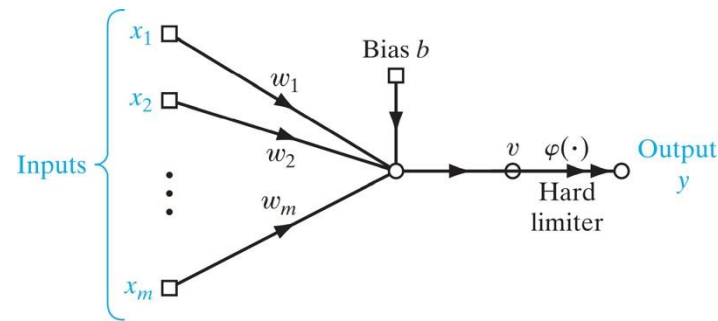
Perceptrons

- Unidade de classificação
- Perceptron de Rosenblatt (1958)
- Cria uma hipersuperfície dentro do espaço das entradas

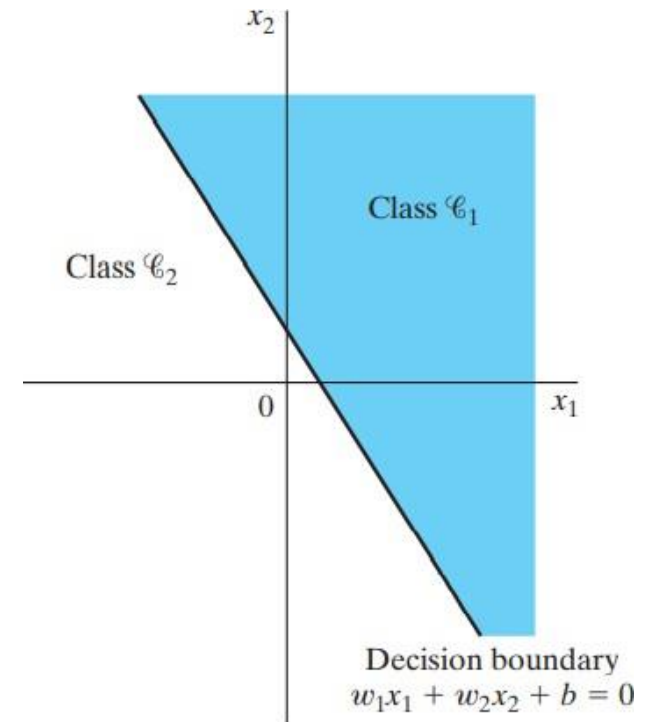


Haykin, S. Neural Networks and Learning Machines - 2009

Perceptrons

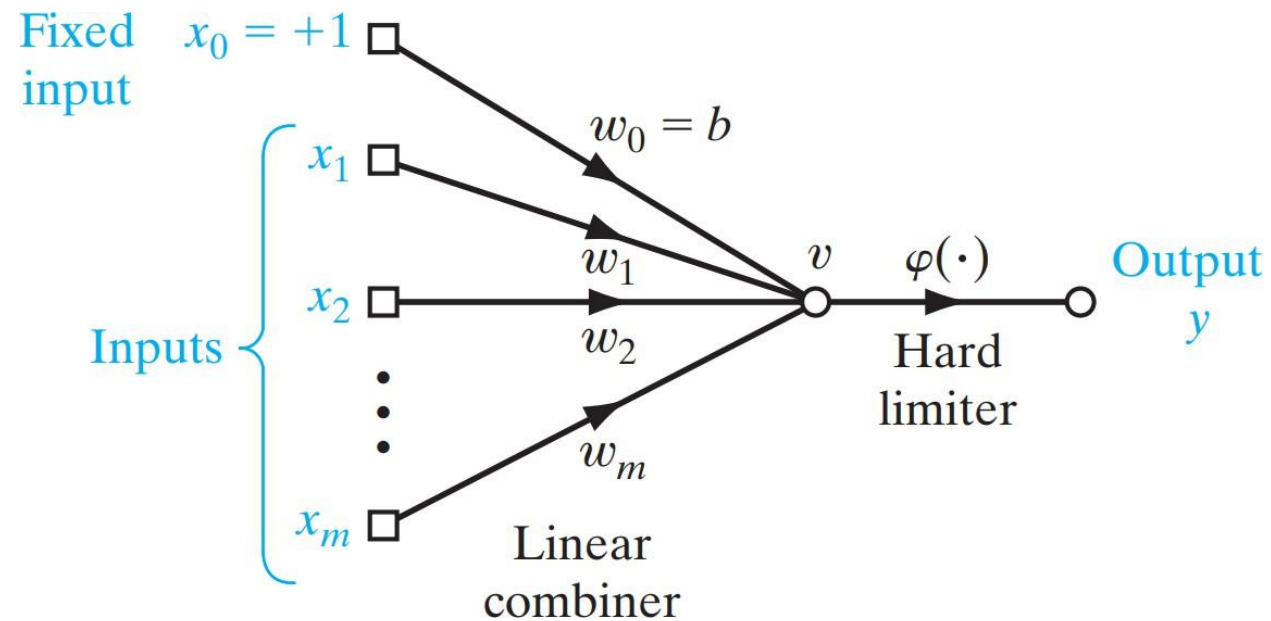


- Unidade de classificação
- Perceptron de Rosenblatt (1958)
- Cria uma hipersuperfície dentro do espaço das entradas
- Na prática, separa as saídas em duas classes: classe 1 $y > 0$, classe 2 $y < 0$



Haykin, S. Neural Networks and Learning Machines - 2009

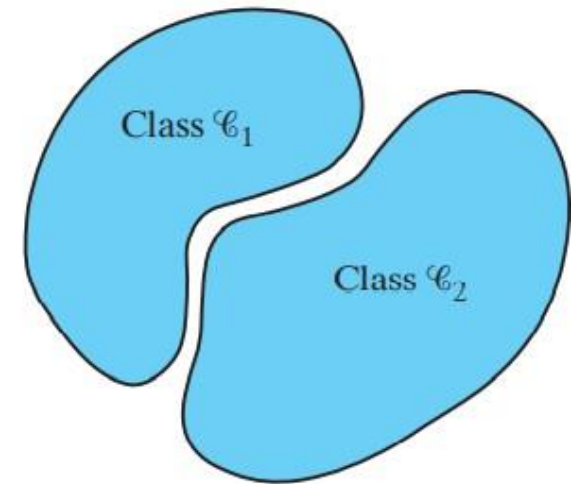
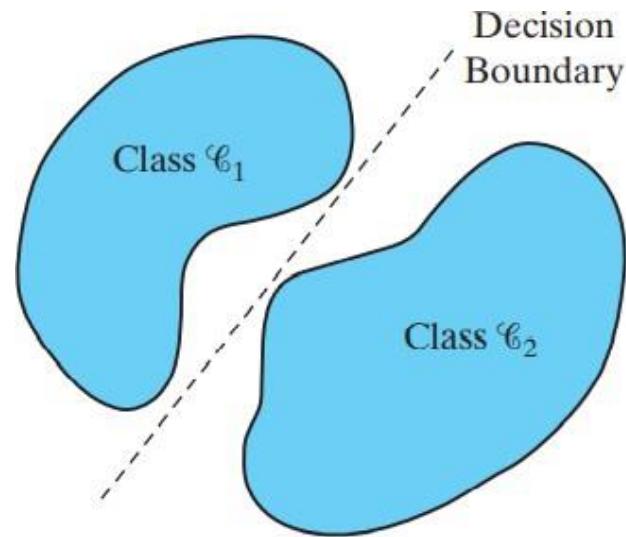
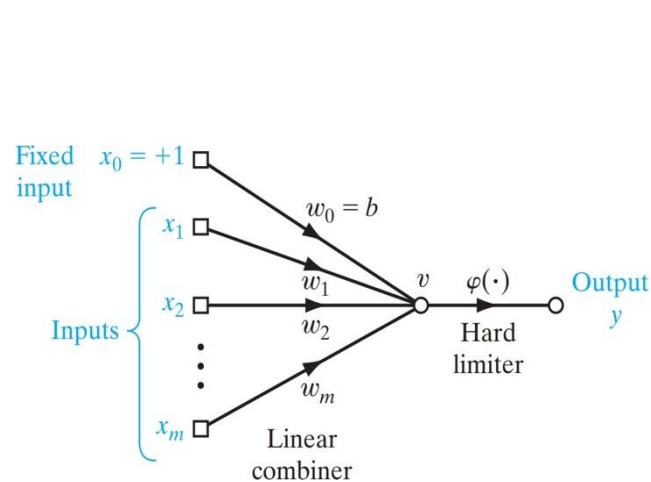
Perceptrons



- Reduzido a uma combinação linear de variáveis

Haykin, S. Neural Networks and Learning Machines - 2009

Perceptrons



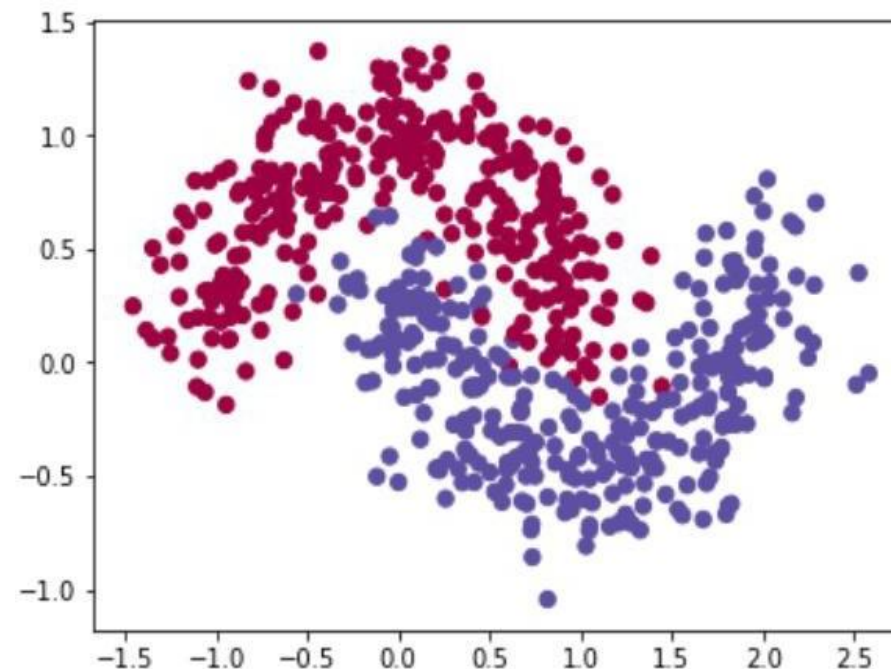
- Limitado a separações mais simples → apenas lineares

Haykin, S. Neural Networks and Learning Machines - 2009

Classificação Binária com RNAs

Classificação binária

- Dados separados em dois grupos distintos
- Cada ponto tem coordenadas (x_1 , x_2)
- Como treinamos uma RNA para separar os grupos?
- Treinamento supervisionado



Classificação binária

Coordenadas (x_1, x_2)

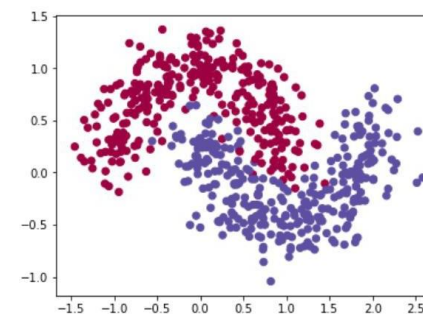


Classe (y)

0 → Azul

1 →
Vermelha

dados de treinamento

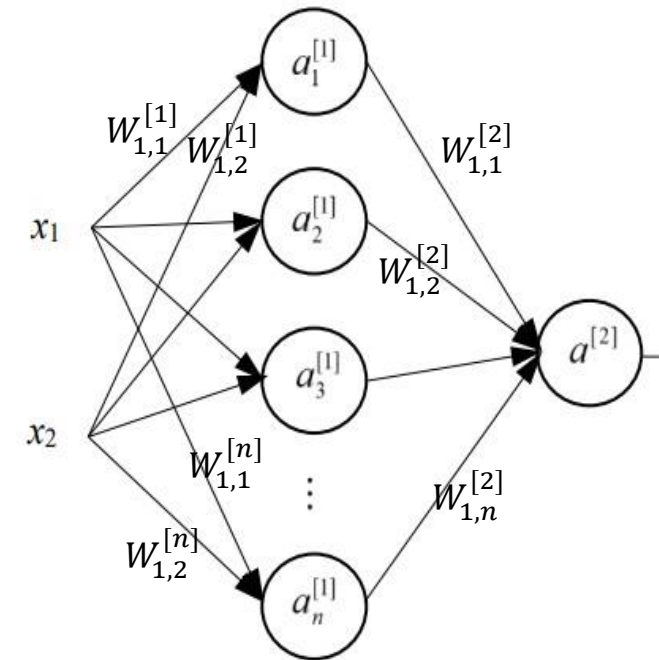


Quem já trabalhou com classificação binária?

- Quais métodos?
- Quais dados?
- Quais problemas?

Exemplo de estrutura de RNA

- Entrada $\rightarrow$ Vetor $\mathbf{x}^{(i)} \in \mathbb{R}^2$
- Saída $\rightarrow \mathbf{y}^{(i)} \in \mathbb{R}$
- Camada intermediária
 - $n^{[1]} = 4$ neurônios
 - $W_{i,j}^{[1]}$ pesos
 - ativação de tangente hiperbólica
- Camada de saída
 - $n^{[2]} = 1$ neurônio
 - $W_{i,j}^{[2]}$ pesos da camada de saída
 - ativação de sigmóide



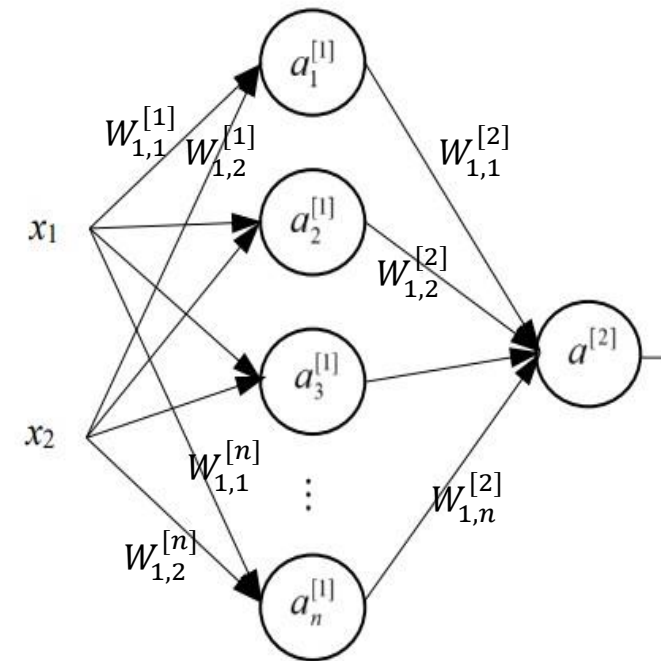
Eduardo Lobo Lustosa Cabral

Propagação para a frente

$$z_1^{[1]} = \mathbf{W}_{1,1}^{[1]}x_1 + \mathbf{W}_{1,2}^{[1]}x_2 + b_1^{[1]}$$

Ou, de forma matricial:

- $\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$
- $\mathbf{a}^{[1]} = \mathbf{g}^{[1]}(\mathbf{z}^{[1]}) \leftarrow$ tangente hiperbólica
- $\mathbf{z}^{[2]} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}$
- $\mathbf{a}^{[2]} = \mathbf{g}^{[2]}(\mathbf{z}^{[2]}) \leftarrow$ sigmóide
- $\hat{\mathbf{y}} = \mathbf{a}^{[2]}$



Propagação para a frente

$$z_1^{[1]} = \mathbf{W}_{1,1}^{[1]}x_1 + \mathbf{W}_{1,2}^{[1]}x_2 + b_1^{[1]}$$

Ou, de forma matricial:

- $\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$
- $\mathbf{a}^{[1]} = \mathbf{g}^{[1]}(\mathbf{z}^{[1]}) \leftarrow$ tangente hiperbólica
- $\mathbf{z}^{[2]} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}$
- $\mathbf{a}^{[2]} = \mathbf{g}^{[2]}(\mathbf{z}^{[2]}) \leftarrow$ sigmóide
- $\hat{\mathbf{y}} = \mathbf{a}^{[2]}$

Variável	Linhas	Colunas
x		
$\mathbf{W}^{[1]}$		
$\mathbf{b}^{[1]}$		
$\mathbf{z}^{[1]}$		
$\mathbf{a}^{[1]}$		
$\mathbf{W}^{[2]}$		
$\mathbf{b}^{[2]}$		
$\mathbf{z}^{[2]}$		
$\mathbf{a}^{[2]}$		

Propagação para a frente: 1 exemplo

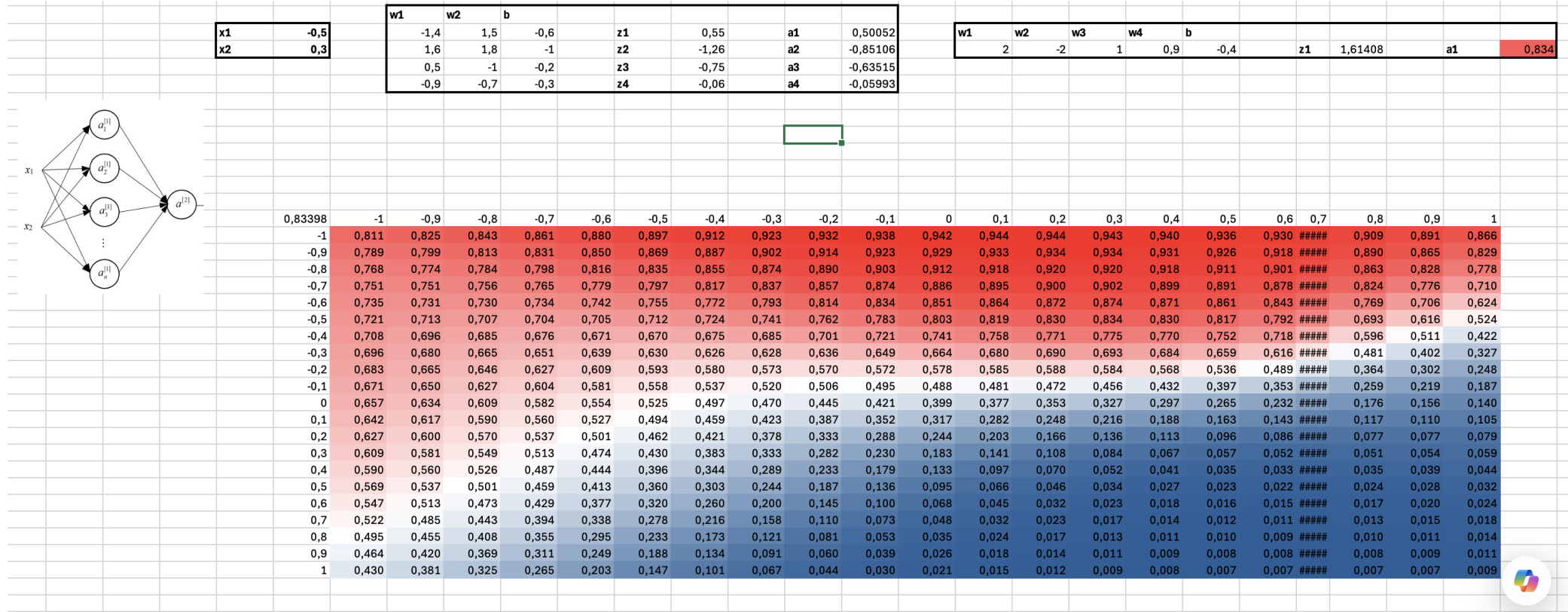
$$z_1^{[1]} = \mathbf{W}_{1,1}^{[1]}x_1 + \mathbf{W}_{1,2}^{[1]}x_2 + b_1^{[1]}$$

Ou, de forma matricial:

- $\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$
- $\mathbf{a}^{[1]} = \mathbf{g}^{[1]}(\mathbf{z}^{[1]}) \leftarrow$ tangente hiperbólica
- $\mathbf{z}^{[2]} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}$
- $\mathbf{a}^{[2]} = \mathbf{g}^{[2]}(\mathbf{z}^{[2]}) \leftarrow$ sigmóide
- $\hat{\mathbf{y}} = \mathbf{a}^{[2]}$

Variável	Linhas	Colunas
x	2	1
$\mathbf{W}^{[1]}$	4	2
$\mathbf{b}^{[1]}$	4	1
$\mathbf{z}^{[1]}$	4	1
$\mathbf{a}^{[1]}$	4	1
$\mathbf{W}^{[2]}$	1	4
$\mathbf{b}^{[2]}$	1	1
$\mathbf{z}^{[2]}$	1	1
$\mathbf{a}^{[2]}$	1	1

Visualizando em Excel



Propagação para a frente

n_s entradas, n_h nós na intermediária

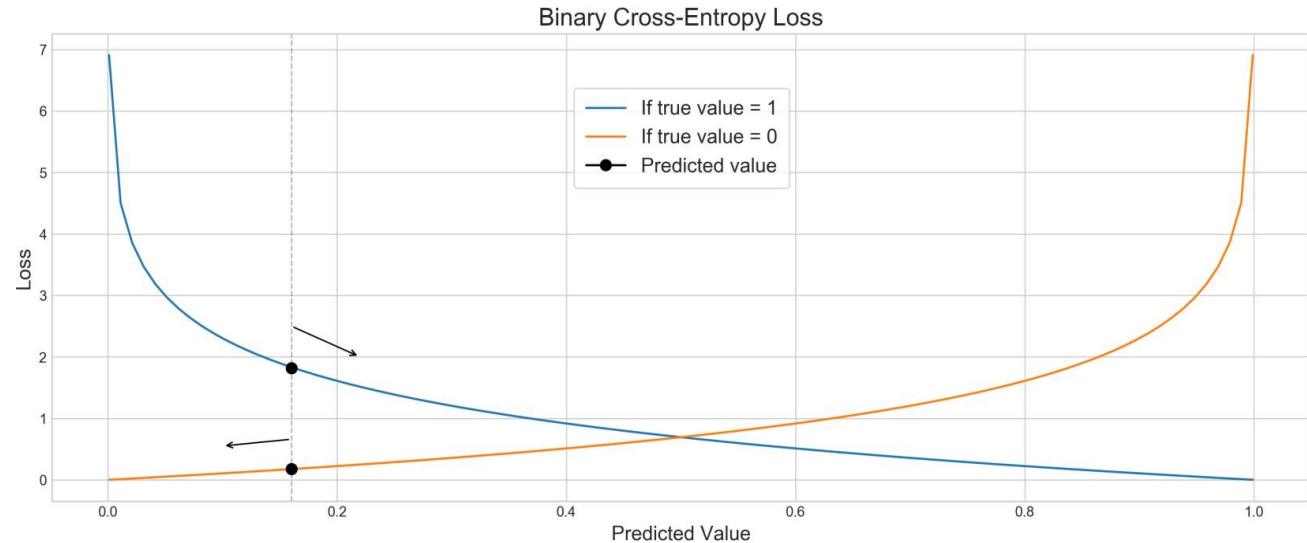
Ou, de forma matricial:

- $\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$
- $\mathbf{a}^{[1]} = \mathbf{g}^{[1]}(\mathbf{z}^{[1]}) \leftarrow$ tangente hiperbólica
- $\mathbf{z}^{[2]} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}$
- $\mathbf{a}^{[2]} = \mathbf{g}^{[2]}(\mathbf{z}^{[2]}) \leftarrow$ sigmóide
- $\hat{\mathbf{y}} = \mathbf{a}^{[2]}$

Variável	Linhas	Colunas
x	2	n_s
$W^{[1]}$	n_h	2
$b^{[1]}$	n_h	1
$z^{[1]}$	n_h	n_s
$a^{[1]}$	n_h	n_s
$W^{[2]}$	1	n_h
$b^{[2]}$	1	1
$z^{[2]}$	1	n_s
$a^{[2]}$	1	n_s

Função de custo (Loss)

- Como comparar a saída da RNA com a “verdade”
- $\hat{y}^{[i]}$ é a previsão da rede no ponto i
- $y^{[i]}$ é o valor de referência no ponto i
- Função de erro logística
- *Loss function*



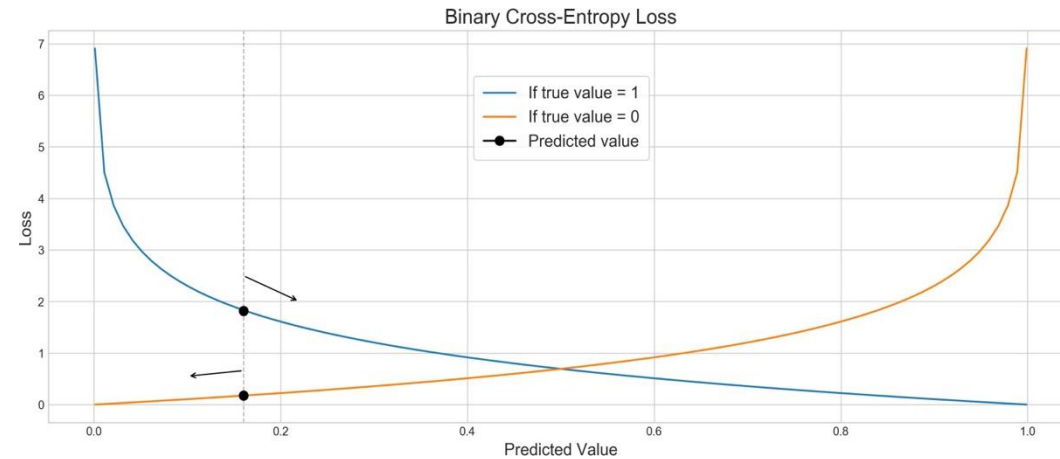
$$L(\hat{y}^{[i]}, y^{[i]}) = -y^{[i]} \log \hat{y}^{[i]} - (1 - y^{[i]}) \log(1 - \hat{y}^{[i]})$$

<https://www.geeksforgeeks.org/machine-learning/ml-cost-function-in-logistic-regression/>

Função de custo (Loss)

- Como comparar a saída da RNA com a “verdade”
- $\hat{y}^{[i]}$ é a previsão da rede no ponto i
- $y^{[i]}$ é o valor de referência no ponto i
- Função de erro logística
- *Loss function*

<https://towardsdatascience.com/logistic-regression-from-scratch-69db4f587e17>



$$L(\hat{y}^{[i]}, y^{[i]}) = -y^{[i]} \log \hat{y}^{[i]} - (1 - y^{[i]}) \log(1 - \hat{y}^{[i]})$$

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{n_s} \sum_{i=1}^{n_s} L(\hat{y}^{[i]}, y^{[i]}) = -\frac{1}{n_s} \sum_{i=1}^{n_s} [y^{[i]} \log \hat{y}^{[i]} + (1 - y^{[i]}) \log(1 - \hat{y}^{[i]})]$$

Como treinar uma RNA?

Devemos encontrar os valores de **W** e **b** que minimizam o erro **$J(W,b)$**

- $z^{[1]} = W^{[1]}x + b^{[1]}$
- $a^{[1]} = g^{[1]}(z^{[1]}) \leftarrow$ tangente hiperbólica
- $z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$
- $a^{[2]} = g^{[2]}(z^{[2]}) \leftarrow$ sigmóide
- $\hat{y} = a^{[2]}$

$$J(W, b) = \frac{1}{n_s} \sum_{i=1}^{n_s} L(\hat{y}^{[i]}, y^{[i]}) = -\frac{1}{n_s} \sum_{i=1}^{n_s} [y^{[i]} \log \hat{y}^{[i]} + (1 - y^{[i]}) \log(1 - \hat{y}^{[i]})]$$

Variável	Linhas	Colunas
x	2	n_s
$W^{[1]}$	n_h	2
$b^{[1]}$	n_h	1
$z^{[1]}$	n_h	n_s
$a^{[1]}$	n_h	n_s
$W^{[2]}$	1	n_h
$b^{[2]}$	1	1
$z^{[2]}$	1	n_s
$a^{[2]}$	1	n_s

Back-propagation

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

NATURE VOL. 323 9 OCTOBER 1986

LETTERS TO NATURE

833

delineating the absolute indigeneity of amino acids in fossils. As AMS techniques are refined to handle smaller samples, it may also become possible to date individual amino acid enantiomers by the ¹⁴C method. If one enantiomer is entirely derived from the other by racemization during diagenesis, the individual D- and L-enantiomers for a given amino acid should have identical ¹⁴C ages.

Older, more poorly preserved fossils may not always prove amenable to the determination of amino acid indigeneity by the stable isotope method, as the prospects for complete replacement of indigenous amino acids with non-indigenous amino acids increases with time. As non-indigenous amino acids undergo racemization, the enantiomers may have identical isotopic compositions and still not be related to the original organisms. Such a circumstance may, however, become easier to recognize as more information becomes available concerning the distribution and stable isotopic composition of the amino acid constituents of modern representatives of fossil organisms. Also, AMS dates on individual amino acid enantiomers may, in some cases, help to clarify indigeneity problems, in particular when stratigraphic controls can be used to estimate a general age range for the fossil in question.

Finally, the development of techniques for determining the stable isotopic composition of amino acid enantiomers may enable us to establish whether non-racemic amino acids in some carbonaceous meteorites²⁷ are indigenous, or result in part from terrestrial contamination.

M.H.E. thanks the NSF, Division of Earth Sciences (grant EAR-8352055) and the following contributors to his Presidential Young Investigator Award for partial support of this research:

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neuron-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of the input units. If the input units are directly connected to the output units it is relatively easy to find learning rules that iteratively adjust the relative strengths of the connections so as to progressively reduce the difference between the actual and desired output vectors². Learning becomes more interesting but

more difficult when we introduce hidden units whose actual or desired states are not specified by the task. (In perceptrons, there are 'feature analysers' between the input and output that are not true hidden units because their input connections are fixed by hand, so their states are completely determined by the input vector; they do not learn representations.) The learning procedure must decide under what circumstances the hidden units should be active in order to help achieve the desired input-output behaviour. This amounts to deciding what these units should represent. We demonstrate that a general purpose and relatively simple procedure is powerful enough to construct appropriate internal representations.

The simplest form of the learning procedure is for layered networks which have a layer of input units at the bottom; any number of intermediate layers; and a layer of output units at the top. Connections within a layer or from higher to lower layers are forbidden, but connections can skip intermediate layers. An input vector is presented to the network by setting the states of the input units. Then the states of the units in each layer are determined by applying equations (1) and (2) to the connections coming from lower layers. All units within a layer have their states set in parallel, but different layers have their states set sequentially, starting at the bottom and working upwards until the states of the output units are determined.

The total input, x_i , to unit j is a linear function of the outputs, y_i , of the units that are connected to j and of the weights, w_{ji} , on these connections

$$x_j = \sum_i y_i w_{ji} \quad (1)$$

Units can be given biases by introducing an extra input to each unit which always has a value of 1. The weight on this extra input is called the bias and is equivalent to a threshold of the opposite sign. It can be treated just like the other weights.

A unit has a real-valued output, y_j , which is a non-linear function of its total input

$$y_j = \frac{1}{1 + e^{-x_j}} \quad (2)$$

Received 19 May, accepted 15 July 1986.

1. Rada, J. I. & Ponsch, R. *Proc. natn. Acad. Sci. U.S.A.* **70**, 1331-1334 (1973).
2. Rada, J. I., Schneider, R. A. & Caprio, G. P. *Science* **186**, 791-793 (1974).
3. Rada, J. I. & Ponsch, R. *Anal. Biochem.* **147**, 437-441 (1985).
4. Weinstein, S. *et al. in Quantitative Dating Methods* (ed. Mahoney, W. C.) 171-193 (Elsevier, Amsterdam, 1984).
5. Engel, M. H., Zambelli, J. E. & Nagi, B. *Anal. Biochem.* **147**, 415-422 (1985).
6. Rada, J. I. *A. Rev. Earth planet. Sci.* **13**, 241-288 (1985).
7. Chisholm, R. S., Nelson, D. E. & Schwagerl, P. *Science* **216**, 1131-1132 (1982).
8. Ambrose, S. H. & DeNiro, M. J. *Nature* **219**, 321-324 (1968).
9. Masako, S. A., Enry, M. L. F., Hare, P. E. & Hoering, T. C. *Yb. Carnegie Inst. Wash.* **82**, 404-410 (1983).
10. Hare, P. E. & Enry, M. L. F. *Yb. Carnegie Inst. Wash.* **82**, 410-414 (1983).
11. Engel, M. H. & Hare, P. E. in *Chemistry and Biochemistry of the Amino Acids* (ed. Barrett, G. C.) 402-419 (Clarendon and Hall, London, 1985).
12. Johnston, R. A. & Rose, M. E. in *Chemistry and Biochemistry of the Amino Acids* (ed. Barrett, G. C.) 402-419 (Clarendon and Hall, London, 1985).
13. Weinstein, S., Engel, M. H. & Hare, P. E. in *Practical Protein Chemistry—A Handbook* (ed. Durrin, A.) 177-184 (Wiley, New York, 1985).
14. Rada, J. I., Gillespie, R., Gieseler, J. A. J. & Hargis, R. E. *Nature* **312**, 442-444 (1984).
15. Mitter, R. M. & Krasinski, N. *Org. Geochem.* **7**, 91-98 (1984).
16. Williams, R. M. & Smith, G. C. *Org. Geochem.* **8**, 91-144 (1977).
17. Engel, M. H. & Hare, P. E. *Yb. Carnegie Inst. Wash.* **81**, 625-630 (1982).
18. Hare, P. E. *Yb. Carnegie Inst. Wash.* **73**, 176-181 (1974).
19. Pilling, C. T. *Nature* **206**, 862 (1965).
20. Neuberg, A. *Adv. Protein Chem.* **4**, 286-383 (1948).
21. Engel, M. H. & Masako, S. A. *Anal. Chem.* **56**, 2559-2600 (1984).
22. Dreyer, G. *Chem. Ber.* **17**, 175-177 (1974).
23. Weinstein, S., Engel, M. H. & Hare, P. E. *Anal. Biochem.* **121**, 370-377 (1982).
24. Masako, S. A., Lee, W. Y. & Hare, P. E. *J. Org. Geochem.* **14**, 145-149 (1982).
25. Masako, S. A., Enry, M. L. F. & Hoering, T. C. *Yb. Carnegie Inst. Wash.* **81**, 413-417 (1982).
26. Naliniy, J. B. *Geochem. Acta* **28**, 117-138 (1964).
27. Engel, M. H. & Nagi, B. *Nature* **296**, 837-840 (1982).

Quem já trabalhou com classificação binária usando RNAs?

- Quais métodos?
- Quais dados?
- Quais problemas?

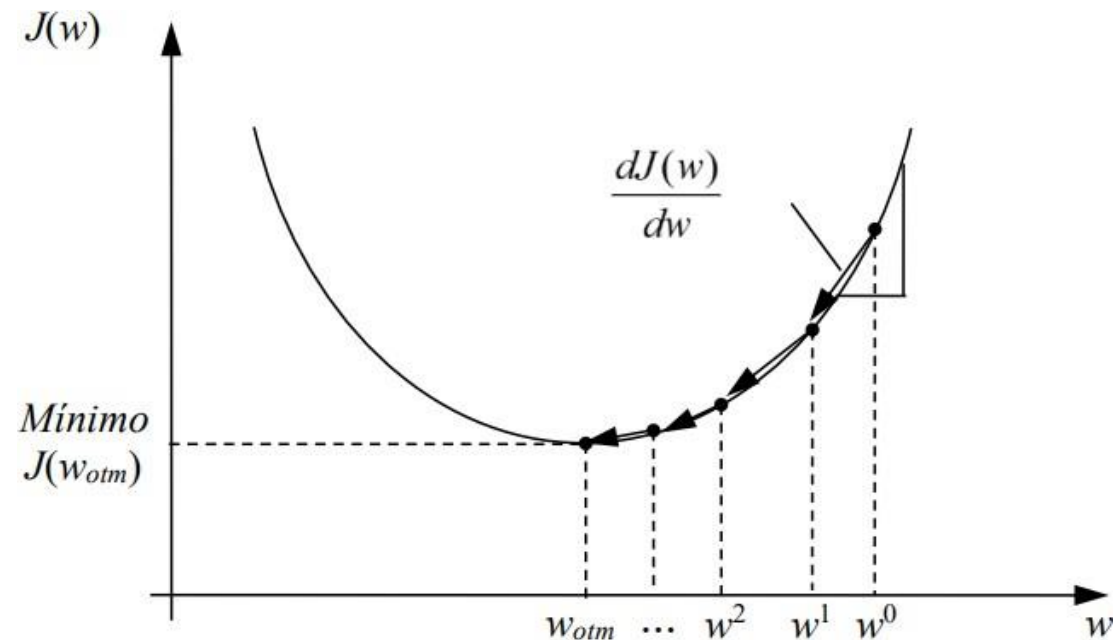
Gradiente Descendente

Gradiente descendente

- Problema de otimização
- Minimizar uma função
- Queremos achar W que minimiza J

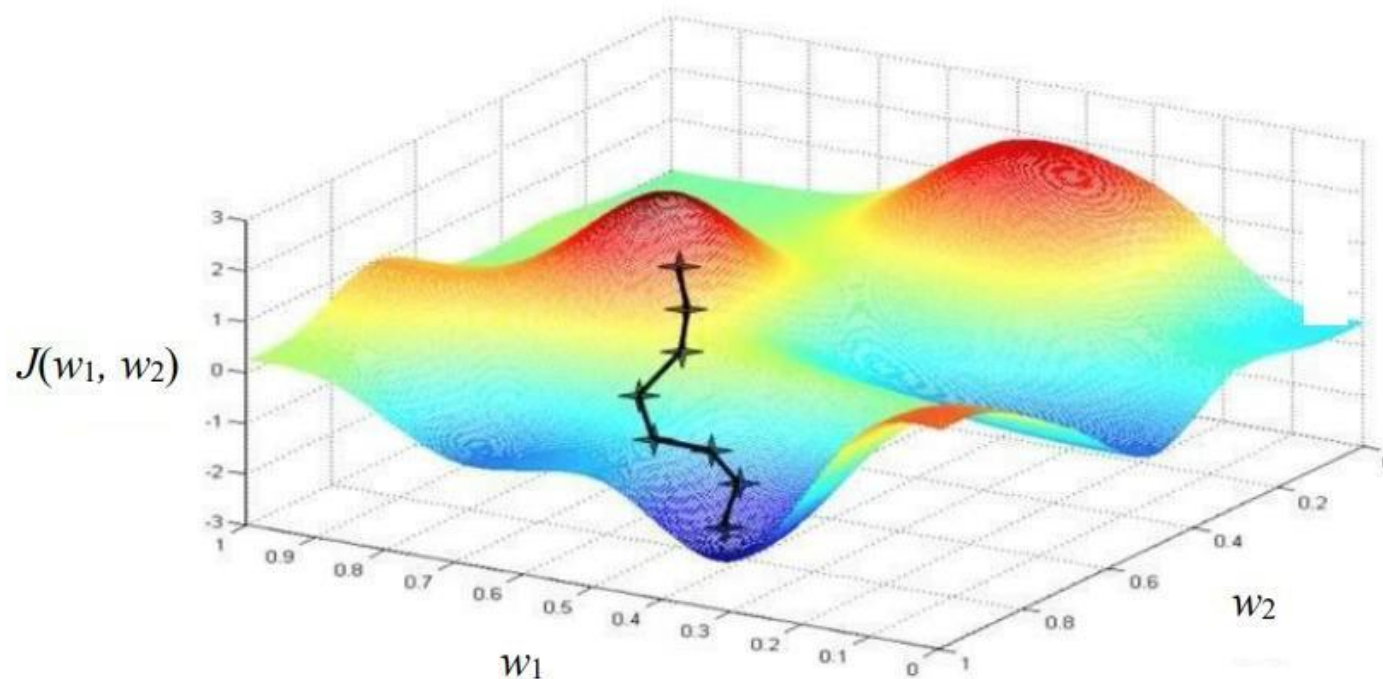
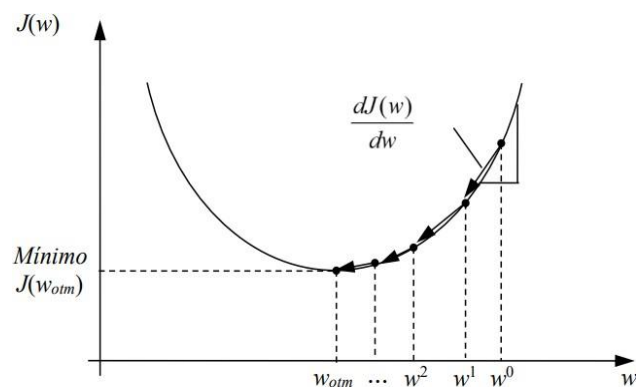
$$\frac{\partial J}{\partial W}$$

- Qual o formato dessa derivada?



Gradiente descendente

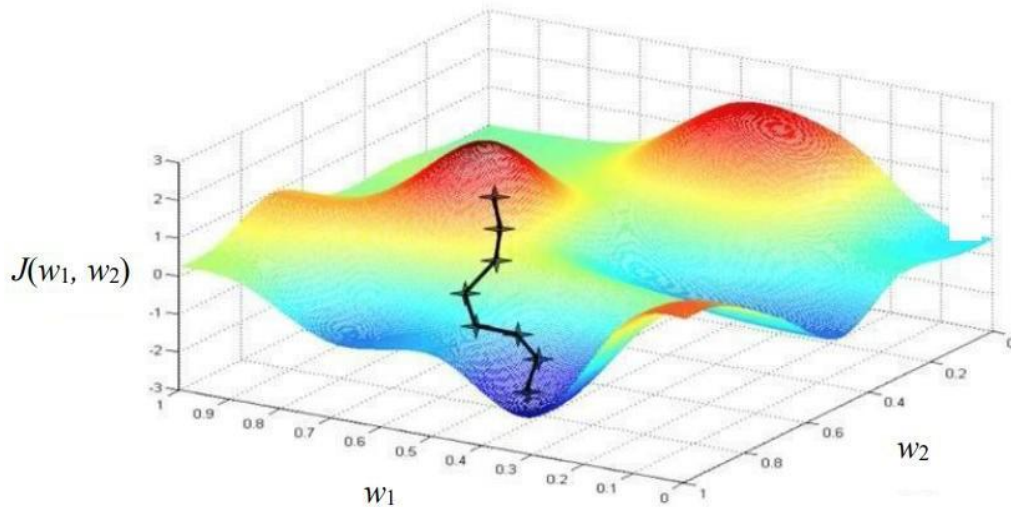
- Problema de otimização
- Minimizar uma função



<https://github.com/RNogales94/ISI-Tensorflow>

Gradiente descendente

- Precisamos do gradiente do custo em relação a cada um dos pesos
- Aqui entra o *backpropagation*



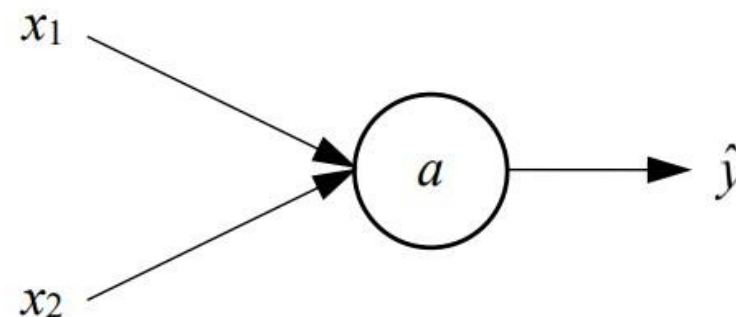
<https://github.com/RNogales94/ISI-Tensorflow>

$$w_{k,j}^{[l]} = w_{k,j}^{[l]} - \alpha \frac{\partial J(\mathbf{W}, \mathbf{B})}{\partial w_{k,j}^{[l]}}, \text{ para } l = 1, \dots, L, k = 1, \dots, n^{[l]}, j = 1, \dots, n^{[l-1]}$$

$$b_k^{[l]} = b_k^{[l]} - \alpha \frac{\partial J(\mathbf{W}, \mathbf{B})}{\partial b_k^{[l]}}, \text{ para } l = 1, \dots, L, k = 1, \dots, n^{[l]}$$

Exemplo de uma RNA de um neurônio

- 2 entradas
- 1 saída
- m exemplos de treinamento
- 1 camada
- 1 neurônio



Exemplo de uma RNA de um neurônio

Propagação para frente

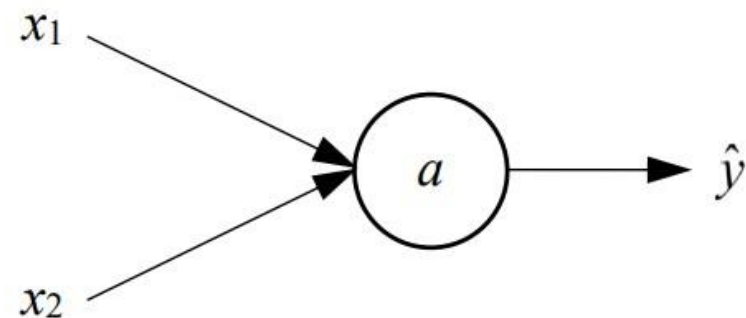
- $z = w_1x_1 + w_2x_2 + b$
- $\hat{y} = a = g(z)$

Parâmetros:

- w_1, w_2, b

Custo:

- $J(w_1, w_2, b) = \frac{1}{m} \sum_{i=1}^m E(\hat{y}^{(i)}, y^{(i)})$



Eduardo Lobo Lustosa Cabral

Exemplo de uma RNA de um neurônio

Propagação para frente

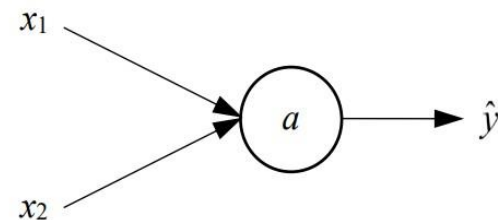
- $z = w_1x_1 + w_2x_2 + b$
- $\hat{y} = a = g(z)$

Parâmetros:

- w_1, w_2, b

Custo:

- $J(w_1, w_2, b) = \frac{1}{m} \sum_{i=1}^m E(\hat{y}^{(i)}, y^{(i)})$



$$\left[\frac{\partial E(a, y)}{\partial a} \right] = \text{derivada da função de erro em relação à saída calculada } (\hat{y} = a);$$

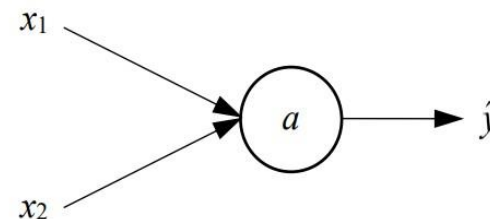
$$\frac{\partial a}{\partial z} = \frac{d[g(z)]}{dz} \Rightarrow \text{derivada da função de ativação } g$$

$$\frac{\partial z}{\partial w_1} = x_1$$

$$\frac{\partial z}{\partial w_2} = x_2$$

$$\frac{\partial z}{\partial b} = 1$$

Exemplo de uma RNA de um neurônio



$$\left[\frac{\partial E(a, y)}{\partial a} \right] = \text{derivada da função de erro em relação à saída calculada } (\hat{y} = a);$$

$$\frac{\partial a}{\partial z} = \frac{d[g(z)]}{dz} \Rightarrow \text{derivada da função de ativação } g$$

$$\frac{\partial z}{\partial w_1} = x_1$$

$$\frac{\partial z}{\partial w_2} = x_2$$

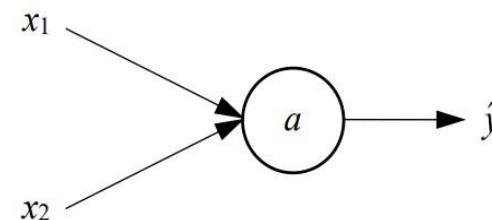
$$\frac{\partial z}{\partial b} = 1$$

$$\frac{\partial E(a, y)}{\partial w_1} = \left[\frac{\partial E(a, y)}{\partial a} \right] \left[\frac{d[g(z)]}{dz} \right] x_1$$

$$\frac{\partial E(a, y)}{\partial w_2} = \left[\frac{\partial E(a, y)}{\partial a} \right] \left[\frac{d[g(z)]}{dz} \right] x_2$$

$$\frac{\partial E(a, y)}{\partial b} = \left[\frac{\partial E(a, y)}{\partial a} \right] \left[\frac{d[g(z)]}{dz} \right]$$

Exemplo de uma RNA de um neurônio



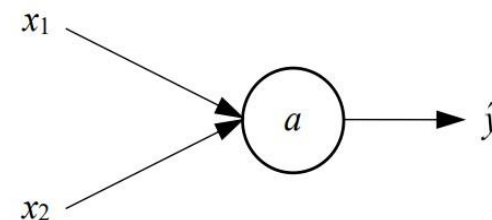
$$\boxed{\frac{\partial E(a, y)}{\partial w_1} = \left[\frac{\partial E(a, y)}{\partial a} \right] \left[\frac{d[g(z)]}{dz} \right] x_1}$$

Para função logística

$$\frac{\partial E(a, y)}{\partial a} = \begin{cases} y = 0 \rightarrow -1/(1 - a) \\ y = 1 \rightarrow -1/a \end{cases}$$

$$L(\hat{y}^{(i)}, y^{(i)}) = -y^{(i)} \log \hat{y}^{(i)} - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

Exemplo de uma RNA de um neurônio

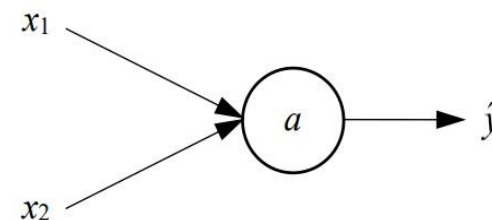


$$\frac{\partial E(a, y)}{\partial w_1} = \left[\frac{\partial E(a, y)}{\partial a} \right] \left[\frac{d[g(z)]}{dz} \right] x_1$$

Para tangente hiperbólica

$$\frac{dg(z)}{dz} = \frac{d}{dz} \left[\frac{e^z - e^{-z}}{e^z + e^{-z}} \right] = \frac{(e^z + e^{-z})^2 - (e^z - e^{-z})^2}{(e^z + e^{-z})^2} = \underbrace{\frac{(e^z + e^{-z})^2}{(e^z + e^{-z})^2}}_1 - \underbrace{\left[\frac{(e^z - e^{-z})}{(e^z + e^{-z})} \right]^2}_{\tanh(z)} = 1 - a^2$$

Exemplo de uma RNA de um neurônio

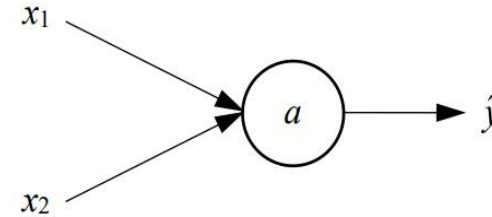


$$\frac{\partial E(a, y)}{\partial w_1} = \left[\frac{\partial E(a, y)}{\partial a} \right] \left[\frac{d[g(z)]}{dz} \right] x_1$$

Para sigmóide

$$\frac{d\sigma(z)}{dz} = \frac{d}{dz} \left[\frac{1}{1+e^{-z}} \right] = \frac{e^{-z}}{(1+e^{-z})^2} = \underbrace{\frac{1}{(1+e^{-z})}}_a \underbrace{\left[1 - \frac{1}{(1+e^{-z})} \right]}_{(1-a)} = a(1-a)$$

Exemplo de uma RNA de um neurônio



$$\frac{\partial E(a, y)}{\partial w_1} = \left[\frac{\partial E(a, y)}{\partial a} \right] \left[\frac{d[g(z)]}{dz} \right] x_1$$

$$\frac{\partial E(a, y)}{\partial w_2} = \left[\frac{\partial E(a, y)}{\partial a} \right] \left[\frac{d[g(z)]}{dz} \right] x_2$$

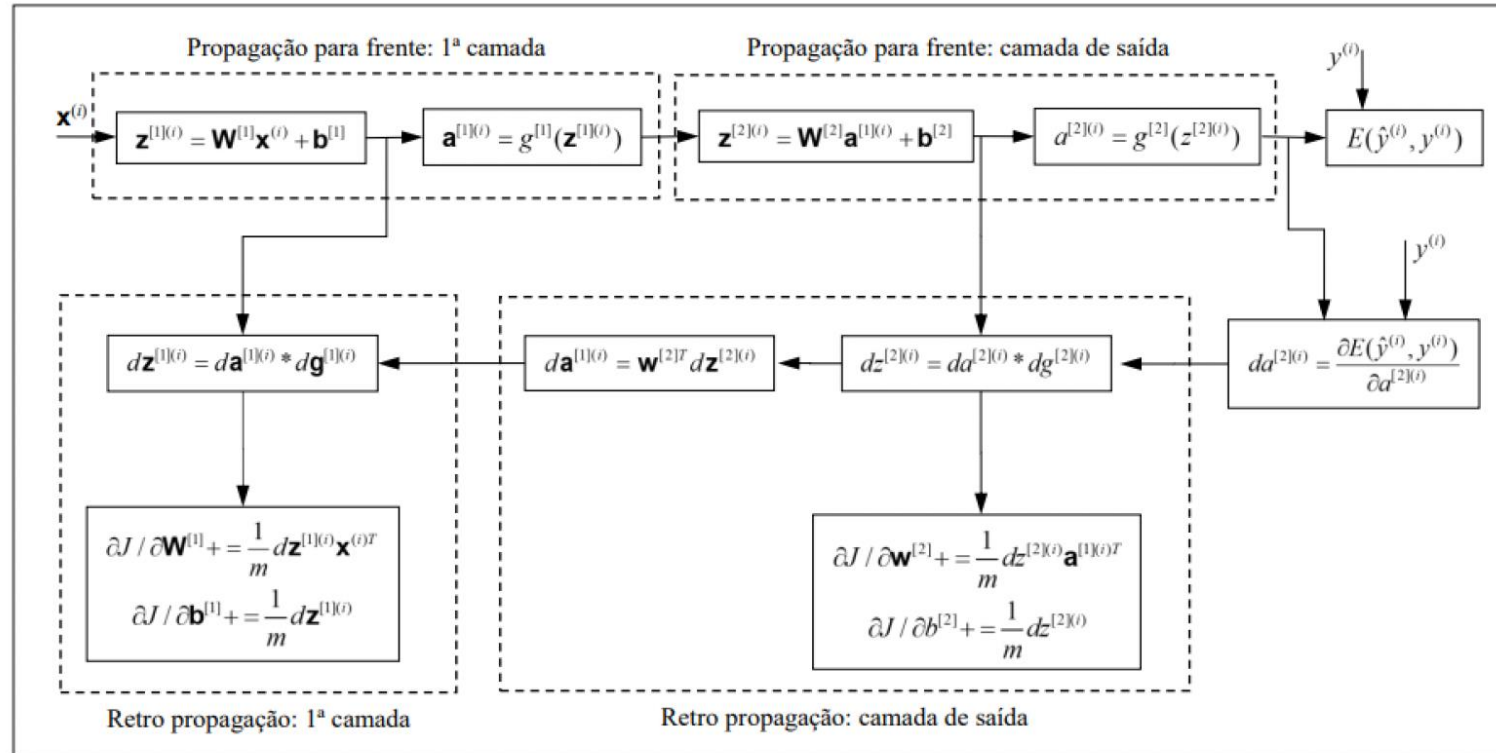
$$\frac{\partial E(a, y)}{\partial b} = \left[\frac{\partial E(a, y)}{\partial a} \right] \left[\frac{d[g(z)]}{dz} \right]$$

Lembrando que:

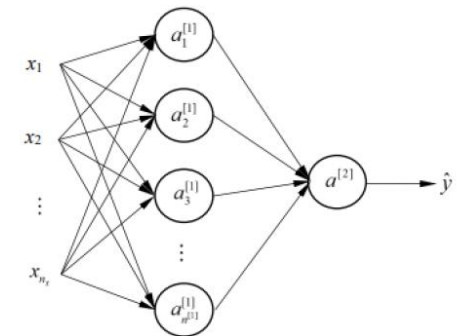
$$J(w_1, w_2, b) = \frac{1}{m} \sum_{i=1}^m E(\hat{y}^{(i)}, y^{(i)})$$

Podemos somar $\frac{\partial E}{\partial w_i}$ para cada ponto e obter $\frac{\partial J(w_1, w_2, b)}{\partial w_i}$

E com mais camadas?



Eduardo Lobo Lustosa Cabral



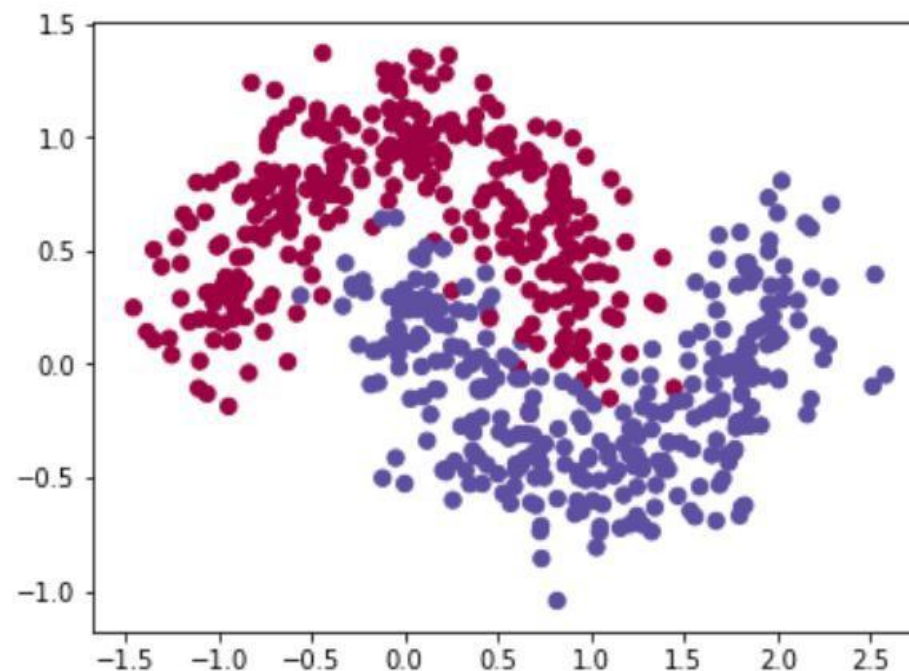
Gradientes e matemáticas

E se eu não sei matemática?

Enunciado Trabalho 1

Trabalho 1

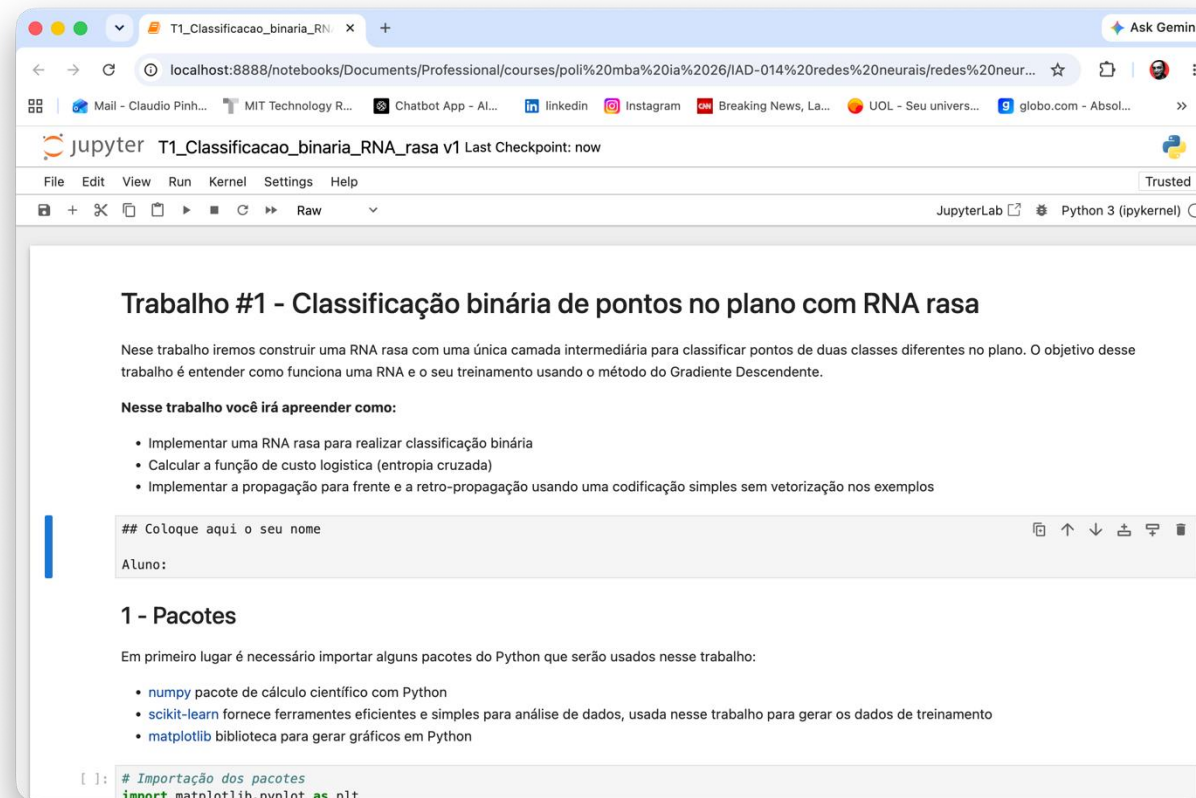
- Classificação binária
- Implementada em um notebook de Python
- **Sem usar bibliotecas de ML**
- Rede neural rasa (1 camada intermediária)
- Sem ajuda de LLMs



Trabalho 1

- Classificação binária
- Implementada em um notebook de Python
- **Sem usar bibliotecas de ML**
- Rede neural rasa
- Sem ajuda de LLMs
- Entregar arquivo ipynb:

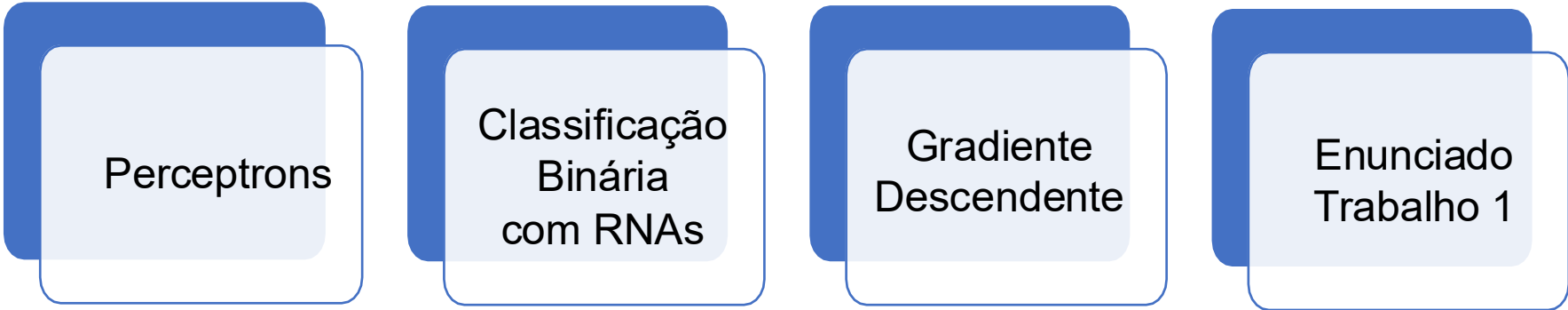
<nome>_<sobrenome>_T1.ipynb



T1_Classificacao_binaria_RNA_rasa v1.ipynb

Encerramento

Aula 2 – Fundamentos de RNAs



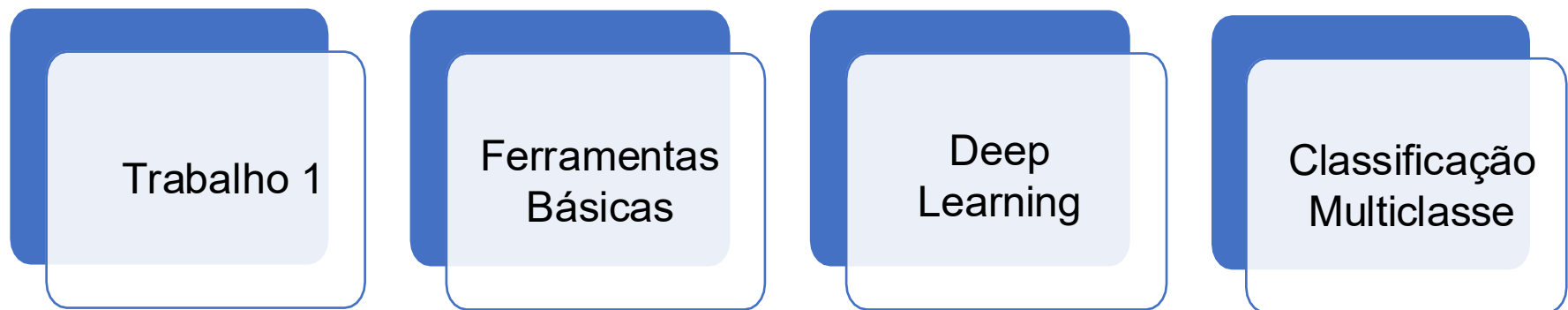
Perceptrons

Classificação
Binária
com RNAs

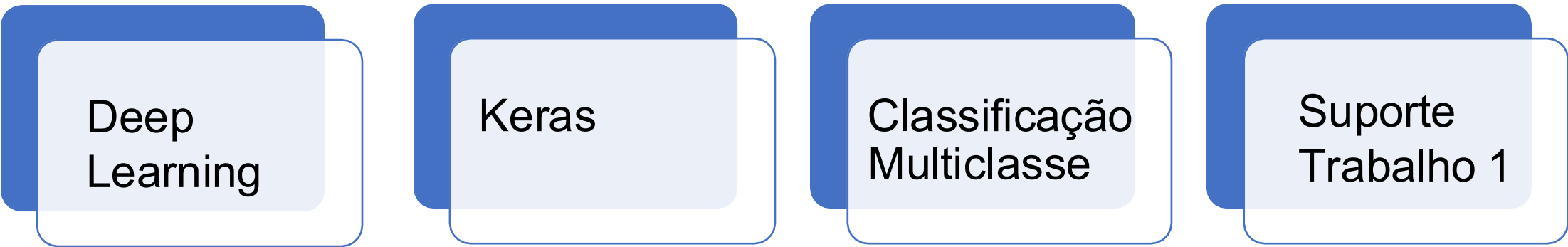
Gradiente
Descendente

Enunciado
Trabalho 1

Próxima Aula – Ferramentas Básicas, Deep Learning e Classificação Multiclasse



Próxima aula – Deep Learning



Deep
Learning

Keras

Classificação
Multiclasse

Suporte
Trabalho 1



IAD-014 Redes Neurais

aula 2

Fundamentos de RNAs

OBRIGADO

Claudio Pinhanez

(a partir de materiais criados por Marlon Sproesser Mathias)